Campus Resource Booking System

Full-Stack Web Development Project Report | Python Flask + MongoDB + Vanilla JS + Docker

Student Name:	Mugunth	**Live Demo:**	campus-resource-booking-cqt7.onrender.com
Branch / Tech:	Computer Science & Engineering	**GitHub Repo:**	github.com/Mugu1825/Campus-Resource-Booking
Deployment:	Docker on Render Cloud	**Database:**	MongoDB Atlas Cloud (Replica Set)

1. Project Motivation & Problem Statement

In our college, booking shared facilities—like computer labs, seminar halls, projectors, or the auditorium—is still handled through physical registers or informal messages. This causes frequent scheduling clashes, confusion over who approved what, and no easy way to check if a lab is free for an upcoming slot.

I built this project to solve that with a clean, fully functional booking website. Rather than making a basic CRUD app with just one generic user type, I designed it around real campus requirements: **3 separate user roles** (Student, Staff, and Admin), **automatic department detection** from roll numbers, **hourly time slot checking**, and **preventing double-bookings** automatically.

2. User Roles & Workflow

Role	How It Works in the App	Key Permissions & Rules
Student	Registers using their college roll number (e.g. 25CSE1011). The app checks the format and automatically detects their department. They can browse labs/halls, check free slots, book, and cancel pending requests.	• Regular booking: Max 4 hours (same day) • Hackathon booking: Up to 48 hours • Can cancel only their own pending requests
Staff	Registers with department. Their account is set to 'pending' until the Admin reviews and approves it. Once active, staff can manage standard labs/rooms and approve student bookings for their department.	• Can approve standard resources • *Cannot* create or approve premium halls or hackathons (preventing misuse)
Admin	The superuser account. Approves new staff accounts, manages all resources (standard and premium), has final say on multi-day hackathons and high-value spaces (like the main auditorium), and can change user roles.	• Full system authority • Approves premium and hackathon bookings • Monitors live stats and user accounts

3. Technology Choices & Architecture

• **Backend (Python & Flask):** I chose Flask because it is lightweight, fast to set up, and makes writing clean REST API routes straightforward. Password security uses flask-bcrypt (salted hashing), and sessions use stateless flask-jwt-extended tokens.
• **Database (MongoDB Atlas):** I used MongoDB because college resources and bookings naturally fit JSON documents. Using Atlas gives a free cloud cluster with automatic TLS encryption and connection pooling.
• **Frontend (Pure HTML5, CSS3, Vanilla JS):** I intentionally avoided bulky frameworks like React or Angular to keep the app fast, clean, and dependency-free. Responsive CSS and modular JavaScript files handle the UI.
• **Containerization (Docker & Nginx):** I packaged both the frontend and backend into a single Docker container. Nginx serves the static HTML/CSS/JS and acts as a reverse proxy forwarding /api/* calls to Gunicorn. This eliminated all CORS headaches and allowed hosting everything on Render's free tier under one single URL.

4. Database Design (Collections)

Collection	Important Fields	Description & Purpose
users	name, email, password, role, account_status, register_no, department	Stores user profiles. Student accounts store roll number (unique). Staff accounts track approval status (pending / active). Passwords are never plain text.
resources	name, type, tier, location, capacity, is_available	Campus assets. tier is either 'standard' (regular labs) or 'premium' (auditorium / compute cluster). is_available allows temporary maintenance takedowns.
bookings	user_id, resource_id, date_start, date_end, start_slot, end_slot, is_hackathon, status	Reservation records. Connects user to resource. Tracks approval status (pending, approved, rejected, cancelled).

5. Key Features & Logic Implemented

A. Roll Number Format Verification:
Students must register with a valid college roll number format: 2-digit year + 3-letter department code + 4 digits (e.g., 25CSE1011). Regex: ^\d{2}(CSE|ECE|EEE|MEE|INT|CIV|CHE|AER|BIO|MBA|MCA)\d{4}$. The backend extracts characters 2-4 and auto-fills their department.

B. Double-Booking Conflict Engine:
Before creating any booking, the backend checks for existing requests on the same resource where status is pending or approved. If dates overlap, it compares slot indices from 09:00 to 21:00. If an overlap is detected, the request is immediately rejected with a clear message.

C. Real-time Slot Availability Grid:
When students click 'Check Availability' on any lab, an interactive visual slot grid opens for the selected date. Free slots appear in green outline, booked slots appear in red, and clicked slots highlight in blue. Clicking 'Book this slot' auto-fills the booking form.

D. Dual Ruleset (Regular vs. Hackathon):
Regular bookings are limited to 4 hours on the same date. For hackathons, students check 'Is Hackathon', enter the event name, and can reserve multi-day blocks up to 48 hours (routed exclusively to Admin for review).

6. Challenges Faced & Solutions

• **CORS & Port Configuration in Deployment:** Initially, running frontend and backend as separate Render services created cross-origin errors and needed two separate URLs. I resolved this by writing a custom Dockerfile with Nginx serving the static frontend and reverse-proxying /api/* to Gunicorn. This gave a unified single-URL deployment with zero CORS issues.
• **MongoDB Eager Index Crash on Startup:** Creating indexes at application startup caused Flask to crash when MongoDB Atlas had not finished connecting. I moved index creation into a one-shot @app.before_request hook so the server starts cleanly even during cold boots.
• **Safe Role Gating:** To prevent staff members from approving high-value auditoriums or overriding multi-day events, I wrote custom Flask decorators (@staff_required and @admin_required) that inspect the JWT token claims before route execution.

7. Conclusion & What I Learned

Working on this project gave me practical, hands-on experience building a real-world web system from start to finish. I learned how to structure REST APIs with Flask blueprints, model relational concepts inside MongoDB, write clean vanilla JavaScript for interactive UI elements, containerize applications using Docker and Nginx, and deploy production code to the cloud.